

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

React III: API Tüketimi ve Yönlendirme

Hafta 7

React III: API Tüketimi ve Yönlendirme

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Hf.1-5 yalnızca tarayıcıda yaşayan bir ön yüz kurdunuz; Hf.6 yalnızca kendi başına çalışan bir sunucu kurdunuz — bu iki parça şimdiye kadar hiç konuşmadı.

- React uygulamanız hâlâ Hf.3'ün `veri.ts` sabit dizisini kullanıyor — sunucunuzdan tamamen habersiz
- `curl`'ün Hf.1'de yaptığı şeyi (bir istek gönderip cevabı okuma) tarayıcı içindeki kodunuzun da yapması gerekiyor
- İki dolu koltuğa aynı anda iki kullanıcı tıklarsa ne olur? Bugüne kadar bu soruyu hiç gerçek bir sunucuya soramadınız

Bu hafta bu iki parça Birleşiyor: gerçek veri, gerçek hatalar, gerçek sayfalar.

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. `fetch`/`Response` ile ağ hatasını 4xx/5xx'ten ayırt eden tiplenmiş bir `apiGet`/`apiPost` istemcisi oluşturmak
2. yükleniyor/hata/veri üç durumlu modelini bir `useVeri<T>` hook'unda uygulayıp yarış durumunu `AbortController` ile önlemek
3. aynı-köken politikasını ve ön uçuş isteğini açıklayıp FastAPI `CORSMiddleware`'i yapılandırmak
4. `react-router` ile parametreli rotalar, ortak sayfa düzeni ve 404 sayfası kurmak
5. Etkinlik/Bilet Sisteminin ön yüzünü Hf.6'nın gerçek API'sine bağlamak

BÖLÜM 1

Ünite 1 — `fetch` ve Tiplenmiş İstemci

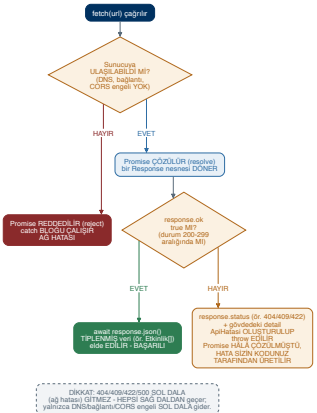
2 • İçerik `fetch`, Response, Ağ Hatası vs 4xx/5xx

- `fetch(url)` bir `Promise<Response>` döndürür — Hf.3 Ünite 4'ün `Promise`/`async`/`await` bilgisi doğrudan kullanılır
- `response.ok` (200–299 ise `true`), `response.status`, `response.json()` — gövdeyi ayrıştırmak ikinci bir `await` gerektirir
- Kritik ayrım: `fetch` yalnızca bir ağ hatasında (sunucuya hiç ulaşılamadı) reddedilir — bir `404`/`409` dönse bile Promise normal şekilde çözülür, yalnızca `response.ok` `false` olur
- `try/catch` bu yüzden `404`'ü yakalamaz — `response.ok` kontrol edilmeden bir hata sessizce "başarılı" gibi işlenir

Tanım: Tiplenmiş istemci (*typed client*)

HTTP isteği, hata kontrolü ve JSON ayrıştırmayı tek bir jenerik fonksiyonda toplayan yardımcı katmandır.

`fetch` Akışı: Ağ Hatası mı, 4xx/5xx mi?



Analoji: Telefonla Sipariş

Analoji

`fetch`'in ağ hatası ile 4xx/5xx ayrımını bir telefonla sipariş vermeye benzetin: numarayı çevirirsiniz (istek); ya karşı taraf hiç açmaz (ağ hatası — arama başarısız sayılır), ya da biri açar ve "siparişiniz alındı" ya da "üzgünüz, o üründen yok" der — ikinci durumda görüşme başarıyla tamamlanmıştır, yalnızca cevap olumsuzdur; telefonu kapatmadan önce karşı tarafın ne söylediğini (durum kodunu) dinlemezseniz ret cevabını kaçırırsınız.

Kod: Tiplenmiş İstemci — `api/istemci.ts`

```ts
// response.ok false olduğunda apiGet/apiPost bu sınıftan bir örnek fırlatır;
// çağıran kod `e instanceof ApiHatasi && e.status === 409` gibi durum
// koduna göre ayrım yapabilir.
export class ApiHatasi extends Error {
  status: number;
  detail: unknown;

  constructor(status: number, detail: unknown) {
    super(`API hatası: ${status}`);
    this.name = 'ApiHatasi';
    this.status = status;
    this.detail = detail;
  }
}
```

`ApiHatasi`, `response.ok` `false` olduğunda `apiGet`/`apiPost` içinde fırlatılır — çağıran kod `status` alanına göre ayrım yapabilir.

Kod: apiGet/apiPost

```
export async function apiGet<T>(
  yol: string,
  { signal }: IstekSecenekleri = {},
): Promise<T> {
  const response = await fetch(`${TABAN_URL}${yol}`, { signal });
  return cevabiIsle<T>(response);
}
```

`apiGet<Etkinlik[]>('/etkinlikler')` çağrısının dönüş tipi Hf.3'ün `interface Etkinlik`'idir — hiçbir dönüşüm kodu yazılmaz, çünkü Hf.6 sunucusu zaten camelCase JSON döner.

BÖLÜM 2

Ünite 2 — Yükleme/Hata/Boş Durumları

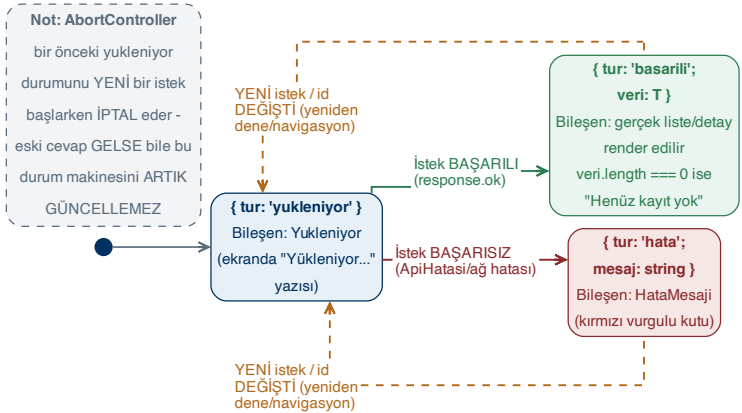
2 • İçerik Üç Durumlu Model, `useVeri<T>`, Yarış Durumu

- `apiGet` çağrısı anında sonuç vermez — bu süre boyunca ekranda ne gösterileceği tasarlanmazsa kullanıcı uygulamanın donduğunu düşünür
- Ayrımcı birleşim tipi: `VeriDurumu<T>` ortak bir `tur` alanıyla `yukleniyor`/`hata`/`basarili` dallarını ayırt eder — Hf.5 Ünite 1'in yarım bıraktığı "API'den veri çekerken de `useEffect` kullanacaksınız" cümlesi bugün gerçekleşiyor
- Yarış durumu: `/etkinlikler/3`'ten `/etkinlikler/7`'ye hızlıca geçildiğinde iki istek yola çıkar — geç gelen cevap ekranı yanlış güncelleyebilir
- `AbortController` + `useEffect` temizleme fonksiyonu (Hf.5'ten tanıdık) önceki isteği iptal eder

Tanım: Yarış durumu (*race condition*)

Birden fazla isteğin cevaplarının beklenmedik sırayla gelip ekranı yanlış güncellemesidir.

Üç Durumlu Veri Modeli



Analoji: Kargo Takip Ekranı ve İki Mektup

Analoji

Üç durumlu veri modelini bir kargo takip ekranına benzetin: paket ”Hazırlanıyor” (yükleniyor), ”Teslim edilemedi” (hata) ya da ”Teslim edildi, içerik burada” (veri) der — ekran her zaman bu üç durumdan birini gösterir, asla boş/belirsiz kalmaz.

Analoji

Yarış durumu ve iptali iki ayrı gişeye art arda gönderdiğiniz iki mektuba benzetin: ikinci mektubun cevabı birincisinden önce gelirse, sonra gelen ilk cevap masanızdaki güncel bilgiyi yanlışlıkla üzerine yazar; bir kurye çağırıp ”ilk mektubu yola çıkarma” demek (iptal) bu karışıklığı önler.

Kod: Ayrımcı Birleşim Tipi — `VeriDurumu<T>`

```
// Ortak bir 'tur' alanıyla ayırt edilen, her dalın FARKLI ek alanlar
// taşıdığı bir discriminated union: durum.tur === 'basarili'
   kontrolünden
// SONRA durum.veri'ye erişmek güvenlidir, 'yukleniyor' dalında durum
   .veri
// HİÇ YOKTUR — TypeScript bunu derleme zamanında denetler.
export type VeriDurumu<T> =
  | { tur: 'yukleniyor' }
  | { tur: 'hata'; mesaj: string }
  | { tur: 'basarili'; veri: T };
```

`durum.tur === 'basarili'` kontrolünden sonra `durum.veri`'ye erişim güvenlidir — `'yukleniyor'` dalında `durum.veri`'ye erişmeye çalışmak derleme hatası verir.

Kod: `useVeri<T>` (1/3) — Kurulum ve Başarılı Yol

```
19 export function useVeri<T>(
20   cagri: (signal: AbortSignal) => Promise<T>,
21   bagimliliklar: unknown[],
22 ): VeriDurumu<T> {
23   const [durum, setDurum] = useState<VeriDurumu<T>>({ tur: 'yukleniyor' });
24
25   useEffect(() => {
26     const controller = new AbortController();
27     setDurum({ tur: 'yukleniyor' });
28
29     // useEffect'in kendisi async OLAMAZ (bir Promise döndüremez); bu
30     // yüzden efekt içinde AYRI bir async fonksiyon tanımlanıp HEMEN
31     // çağrılır.
32     async function veriyiGetir() {
33       try {
34         const veri = await cagri(controller.signal);
35         setDurum({ tur: 'basarili', veri });
```

`useEffect`'in kendisi `async` olamaz; bu yüzden içinde ayrı bir `async` fonksiyon tanımlanıp hemen çağrılır.

Kod: useVeri<T> (2/3) — İptal Edilen İstek

```
36      } catch (hata) {
37        if (hata instanceof DOMException && hata.name === 'AbortError') {
38          // İstek bilinçli olarak iptal edildi (yeni bağımlılık ya da
39          // unmount) — state'i GÜNCELLEMEDEN sessizce çık.
40          return;
41        }
42        const mesaj =
43          hata instanceof ApiHatasi
44            ? typeof hata.detail === 'string'
45              ? hata.detail
46              : hata.message
47            : hata instanceof Error
48              ? hata.message
49              : 'Bilinmeyen bir hata oluştu.';
```

`AbortError` yakalanırsa state hiç güncellenmez — bu, iptal edilmiş bir isteğin cevabının ekranı yanlış göstermesini önler.

Kod: useVeri<T> (3/3) — Temizleme ve Dönüş

```
50        setDurum({ tur: 'hata', mesaj });
51      }
52    }
53
54    veriyiGetir();
55
56    return () => controller.abort();
57    // eslint-disable-next-line react-hooks/exhaustive-deps
58  }, bagimliliklar);
59
60  return durum;
61 }
```

Temizleme fonksiyonu `controller.abort()` çağırır — eski cevap gelse bile `AbortError` yüzünden state güncellenmez.

BÖLÜM 3

Ünite 3 — CORS İlk Temas

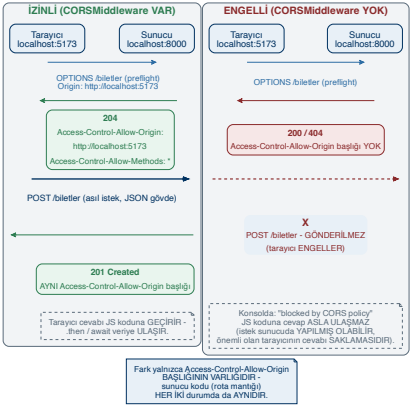
2 • İçerik Köken, Aynı-Köken Politikası, Ön Uçuş İsteği

- Hf.6'nın bıraktığı cümle gerçekleşiyor: ilk `fetch` denemenizde tarayıcı konsolu kırmızı bir "CORS policy" hatasıyla dolar
- Köken (origin): protokol+alan adı+port üçlüsü — `http://localhost:5173` ile `http://127.0.0.1:8000` farklı iki köken'dir (yalnızca port farkı bile yeterlidir)
- Aynı-köken politikası: tarayıcının varsayılan davranışı, farklı köken'e giden bir isteğin cevabını JS koduna okutturmamasıdır — bir güvenlik önlemidir
- Ön uçuş `OPTIONS` isteği: JSON gövdeli `POST`/`PATCH`/`DELETE` gibi "basit olmayan" istekler için, asıl istekten önce tarayıcı "izin var mı?" diye sorar

Tanım: Ön uçuş isteği (*preflight request*)

Basit olmayan bir isteğin gönderilmeden önce izin sormak için yollanan `OPTIONS` isteğidir.

CORS Ön Uçuş Akışı: İzinli vs Engellenen



Analoji: Apartman Kapıcısı ve Kuryenin Notu

Analoji

Aynı-köken politikasını bir apartmanın kapıcısının varsayılan olarak yalnızca kendi apartmanından gelen taleplere kapı açmasına benzetin: başka bir binadan (farklı köken) gelen bir kuryeyi, siz izin vermediğiniz sürece içeri almaz.

Analoji

Ön uçuş `OPTIONS` isteğini bir kuryenin kapıya gelmeden önce yönetime bir not bırakıp ”böyle bir paket getirebilir miyim?” diye sormasına benzetin: yönetim (sunucu) ”evet, bu bina adına izin var” diye cevap verirse kurye asıl paketi getirir.

Kod: `CORSMiddleware` — `app/main.py`

```
22 app.add_middleware(
23     CORSMiddleware,
24     allow_origins=["http://localhost:5173"],
25     allow_credentials=True,
26     allow_methods=["*"],
27     allow_headers=["*"],
28 )
```

Bu haftanın sunucu tarafındaki tek ekleme — `allow_origins` yalnızca geliştirme adresini içerir; `["*"]` (herkese izin) üretimde güvenlik riski taşır (Hf.11).

Kod: CORS Başlık Testi — `test_cors.py`

```python
def test_cors_izinli_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:5173"}
    )
    assert cevap.status_code == 200
    assert cevap.headers["access-control-allow-origin"] == "http://
        localhost:5173"
```

İzinli köken (`localhost:5173`) ile `200` döner ve `access-control-allow-origin` başlığı o kökeni taşır; ikinci test (`test_cors_izinsiz_origin`) izinsiz kökende (`localhost:4000`) yine `200` ama başlığın eksik olduğunu doğrular.

BÖLÜM 4

Ünite 4 — React Router: Sayfalar, Parametreli Rotalar

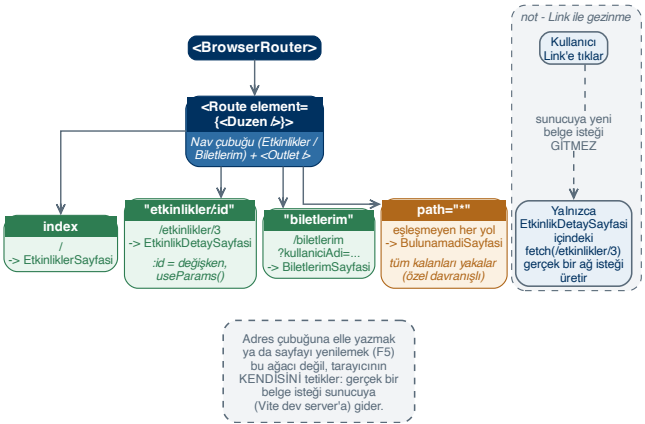
2 • İçerik BrowserRouter, Route, Link, Outlet

- Hf.4-6 boyunca tüm içerik tek bir `App` bileşeninin içinde, tek bir URL'de (`/`) yaşadı — adres çubuğu hiçbir zaman değişmedi
- `<Route path="/etkinlikler/:id" element={...} />` + `useParams()` — URL'nin değişken bir parçasını yakalar
- `<Link to="...">`: tıklandığında sunucuya yeni bir HTTP isteği göndertmez, yalnızca adres çubuğunu günceller
- `Duzen+Outlet`: nav çubuğunu taşıyan sarmal bileşen — `Outlet`, eşleşen alt rotanın render edileceği yer tutucudur
- `path="*"`: hiçbir yolla eşleşmeyen adresler için 404 sayfası

Tanım: Yer tutucu çıkış noktası (*outlet*)

Layout içinde eşleşen alt rotanın render edileceği konumu işaretleyen bileşendir.

URL → Route → Sayfa: Router Ağacı



Analoji: AVM Tabelası, Asansör ve Dergi Başlık Şeridi

Analoji

`Route` eşlemesini bir alışveriş merkezinin girişindeki kat yönlendirme tabelasına benzetin: "1. Kat → Giyim" yazar; hangi kata gitmek istediğinizi (URL) söylediğinizde doğru mağazaya (bileşene) yönlendirilirsiniz.

Analoji

Layout/`Outlet`'i bir derginin her sayfasında aynı kalan başlık/dipnot şeridine benzetin: üstteki logo ve alttaki sayfa numarası (nav çubuğu) hiç değişmez, yalnızca ortadaki makale içeriği (Outlet) sayfadan sayfaya değişir.

Kod: Ortak Sayfa Düzeni — `Duzen.tsx`

```tsx
export function Duzen() {
  return (
    <div className="duzen">
      <nav className="duzen-nav">
        <span className="duzen-baslik">KTÜ Etkinlik ve Bilet Sistemi</span>
        <Link to="/">Etkinlikler</Link>
        <Link to="/biletlerim">Biletlerim</Link>
      </nav>
      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

`Route`'un `element` prop'una verilen `Duzen` içine yerleştirilen iç `Route`'lar hep aynı nav çubuğunun altında görünür.

Kod: Rota Ağacı — `App.tsx`

```tsx
export function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route element={<Duzen />}>
          <Route index element={<EtkinliklerSayfasi />} />
          <Route path="etkinlikler/:id" element={<EtkinlikDetaySayfasi />} />
          <Route path="biletlerim" element={<BiletlerimSayfasi />} />
          <Route path="*" element={<BulunamadiSayfasi />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

`Routes` içindeki `Route`'lar sırayla değerlendirilir, ilk eşleşen yol render edilir — `path="*"` en sonda tanımlanır.

BÖLÜM 5

Özet

Özet

Özet

- `fetch` + `ApiHatasi`: ağ hatasını 4xx/5xx'ten ayıran tiplenmiş bir `apiGet`/`apiPost` istemcisi kurdunuz
- `VeriDurumu<T>` + `useVeri<T>` + `AbortController`: yükleniyor/hata/veri durumlarını yönettiniz, yarış durumunu önlediniz
- `CORSMiddleware`: aynı-köken politikasını ve ön uçuş isteğini aştınız, tarayıcı artık sunucunuzun cevabını okuyabiliyor
- `react-router`: uygulamanızı `Duzen`+`Outlet` altında gerçek, adres çubuğunda görünen sayfalara böldünüz

5 • İzleme Aktarım — Başka Nerede?

- `apiGet`/`apiPost`/`ApiHatasi` deseni dönem boyunca her yeni API çağrısında (Hf.10'un giriş isteği, Hf.13'ün WebSocket öncesi bağlantı isteği) aynen yeniden kullanılacak
- Üç durumlu model Hf.10'un giriş formunda, Hf.13'ün WebSocket bağlantı durumunda aynı iskeletle tekrar karşınıza çıkacak
- `allow_origins` listesi Hf.14'te gerçek üretim alan adınıza güncellenecek; Hf.11 CORS'u bir güvenlik konusu olarak daha derin ele alacak
- `Duzen`+`Outlet` iskeleti Hf.10'da `/giris`/ `/panel` rotalarıyla genişleyecek
- Bugün Etkinlik/Bilet Sisteminin ön yüzü nihayet Hf.6'nın gerçek sunucusuna bağlandı — ama sunucunun belleği hâlâ geçicidir, gelecek hafta bu veriyi kalıcı hâle getireceksiniz

Sırada Ne Var?

Bu hafta lab'da

`src/api/` istemcisini, `useVeri` hook'unu ve `App.tsx`'in router kurulumunu kendi elinizle yazıp iki terminal (`uvicorn app.main:app --reload` + `npm run dev`) açık tutarak Etkinlik/Bilet Sisteminin ön yüzünü gerçek sunucuya bağlayacak, koltuk çakışması `409`'unu ekranda göreceksiniz.

- Kendinizi Sınayın her üniteyi kapsıyor: bir `404` cevabı `fetch`'in Promise'ini reddeder mi, `AbortController` olmasaydı hangi sırayla gelen cevaplar ekranı yanlış gösterirdi, tarayıcı ilk olarak hangi isteği gönderir ve neden, `F5` yenilemesini React Router mı sunucu mu karşılar
- Gelecek hafta: Veritabanı — SQLAlchemy ve Kalıcılık — sunucunuzun belleği artık gerçek bir veritabanına dönüşecek